

Universidad Mesoamericana
Facultad de Ingeniería
Desarrollo Web II Asincrónico
Ing. Jerson Quiñónez



RESUMEN AGOSTO 2026

Diego Alejandro Bran Valdéz

202460510

Técnico en Desarrollo de Software

Guatemala, septiembre de 2026

Arquitectura Modelo-Vista-Controlador (MVC) en Backend

La implementación de un backend robusto requiere una arquitectura que separe las responsabilidades para facilitar la escalabilidad y el mantenimiento. En los ejercicios desarrollados (como el API de concesionario de vehículos), se prioriza la estructura MVC enfocada en backend, dividida en las siguientes capas estructurales:

- Rutas (Routes): Encargadas de interceptar las peticiones HTTP (endpoints) de los clientes y redirigirlas al controlador adecuado.
- Controladores (Controllers): Manejan la lógica de la petición y la construcción de la respuesta. Extraen los datos del cuerpo (body), parámetros (params) o query strings, y llaman a los servicios correspondientes para procesar la información.
- Servicios (Services): Contienen la lógica de negocio pura. Se separan del controlador para que el código sea reutilizable y más fácil de someter a pruebas unitarias.
- Capa de Datos (Mock-Data / Models): Gestionan el almacenamiento de información, ya sea simulando el entorno mediante archivos JSON (Mock Data de hasta 10 documentos por colección) o modelos conectados a bases de datos reales.

Métodos HTTP y Manejo de Status Codes

Para la construcción correcta de una API RESTful, se deben utilizar los métodos estándar definidos por el protocolo HTTP para realizar operaciones de tipo CRUD. Además, es obligatorio acompañar cada respuesta con un *Status Code* adecuado para informar sobre el resultado de la operación.

Método HTTP	Operación Asignada	Status Code Exitoso	Status Code de Error Frecuente
GET	Lectura (Obtener o colecciones documentos)	200 (OK)	404 (Not Found), 500 (Internal Error)
POST	Creación (Insertar nuevos registros)	201 (Created)	400 (Bad Request)
PUT	Actualización (Modificar registros existentes)	200 (OK)	400, 404 (Not Found)
DELETE	Eliminación (Borrar registros)	200 (OK) o 204 (No Content)	404 (Not Found)

Validaciones de Datos en Endpoints

Un componente clave para la consistencia y seguridad del sistema es la validación estricta de las entradas (Payloads JSON) antes de guardarlas, especialmente en métodos POST y PUT:

- **Requerimiento de Campos:** Diferenciar qué información es estrictamente obligatoria (ej. primer_nombre, username, password) frente a la opcional (ej. segundo_nombre).
- **Longitud y Formato:** Verificar las limitantes, tales como nombres comerciales con máximo de 50 caracteres o teléfonos limitados a un tamaño exacto de 8 dígitos.
- **Reglas Complejas:** Uso de expresiones regulares para garantizar emails válidos o passwords seguras (las cuales deben requerir longitud de 8 a 25 caracteres, y al menos un símbolo, un dígito, una mayúscula y una minúscula).

MongoDB: Buenas Prácticas de Bases de Datos

De cara a la escalabilidad de nuestras aplicaciones en Node.js con MongoDB, es vital implementar directrices estandarizadas al modelar datos NoSQL:

1. **Referencias vs. Datos Embebidos:** Los datos se deben *embeber* cuando están fuertemente relacionados y se consultan juntos. Por el contrario, se deben utilizar *referencias* cuando los documentos se reutilizan en múltiples lugares o cuando crecen de manera independiente (evitando el anidamiento infinito). Asimismo, no se debe manejar almacenamiento directo de archivos dentro de las colecciones.
2. **Aprovechar el ObjectId:** Se debe delegar y aprovechar la generación del campo `_id` único por parte de MongoDB, el cual ayuda a identificar documentos inequívocamente y funciona como un índice principal.
3. **Tamaño de Documentos:** Existe un límite físico estipulado por MongoDB de 16MB por documento BSON. El diseño del esquema debe contemplar que los arreglos y objetos anidados no incrementen inmensamente este tamaño, puesto que complica el rendimiento de las actualizaciones y el ancho de banda del servidor.